

Interraction avec l'utilisateur

Table des matières

Exemple complet	2
États	4
Associer un comportement à un composant	5
Bonnes pratiques de codage	5
Débogueur	7
Exécuter une application avec le débogueur	7
Utiliser le débogueur	8
Exercices	9
Références	9

Dans ce chapitre, le concept d'état est présenté avant d'associer un comportement à une zone de texte et à un bouton. De plus l'utilisation du débogueur ainsi de de bonne pratique de programmation sont expliqués.

Exemple complet

Reprenons l'exemple du cours précédent: Le convertisseur de pouces à centimètre. Quelques modifications ont été apporté à l'interface.

```
1 package com.example.convertisseur1
2
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.compose.foundation.clickable
7 import androidx.compose.foundation.layout.Arrangement
8 import androidx.compose.foundation.layout.Column
9 import androidx.compose.foundation.layout.Spacer
10 import androidx.compose.foundation.layout.fillMaxSize
11 import androidx.compose.foundation.layout.height
12 import androidx.compose.foundation.layout.padding
13 import androidx.compose.material.icons.Icons
14 import androidx.compose.material.icons.filled.Clear
15 import androidx.compose.material3.Button
16 import androidx.compose.material3.ExperimentalMaterial3Api
17 import androidx.compose.material3.Icon
18 import androidx.compose.material3.MaterialTheme
19 import androidx.compose.material3.Surface
20 import androidx.compose.material3.Text
21 import androidx.compose.material3.TextField
22 import androidx.compose.runtime.Composable
23 import androidx.compose.runtime.mutableStateOf
24 import androidx.compose.runtime.saveable.rememberSaveable
25 import androidx.compose.ui.Alignment
26 import androidx.compose.ui.Modifier
27 import androidx.compose.ui.graphics.Color
28 import androidx.compose.ui.tooling.preview.Preview
29 import androidx.compose.ui.unit.dp
30 import androidx.compose.ui.unit.sp
31 import com.example.convertisseur1.ui.theme.Convertisseur1Theme
32
33 class MainActivity : ComponentActivity() {
34     override fun onCreate(savedInstanceState: Bundle?) {
35         super.onCreate(savedInstanceState)
36         setContent {
37             Convertisseur1Theme {
38                 // A surface container using the 'background' color from the
theme
39                 Surface(
40                     modifier = Modifier.fillMaxSize(),
41                     color = MaterialTheme.colorScheme.background
42                 ) {
```

```

43         Convertisseur("Pouce à cm")
44     }
45 }
46 }
47 }
48 }
49
50 @ExperimentalMaterial3Api
51 @Composable
52 fun Convertisseur(name: String, modifier: Modifier = Modifier) {
53     var resultat = rememberSaveable { mutableStateOf("") }
54     var entree = rememberSaveable { mutableStateOf("") }
55     Column(
56         modifier = modifier.fillMaxSize(),
57         verticalArrangement = Arrangement.Top,
58         horizontalAlignment = Alignment.CenterHorizontally
59     ){
60         Text(
61             text = "$name",
62             fontSize = 36.sp,
63             modifier = Modifier.padding(
64                 start = 16.dp,
65                 top = 16.dp,
66                 end = 16.dp,
67                 bottom = 32.dp
68             )
69         )
70     }
71
72     TextField(
73         value = entree.value,
74         onValueChange = { entree.value = it },
75         label = { Text(text = "Pouces: ") },
76         trailingIcon = {
77             Icon(
78                 Icons.Default.Clear,
79                 contentDescription = "clear text",
80                 modifier = Modifier
81                     .clickable {
82                         //text = TextFieldValue("")
83                         entree.value = ""
84                     }
85             )
86         },
87     )
88
89     Spacer(Modifier.height(16.dp))
90     Button(onClick = {
91         var pouce = entree.value.toDouble()
92         var centimetre = pouce * 2.54
93         resultat.value = "$centimetre"
94     }){
95         Text(text = "Convertir")
96     }

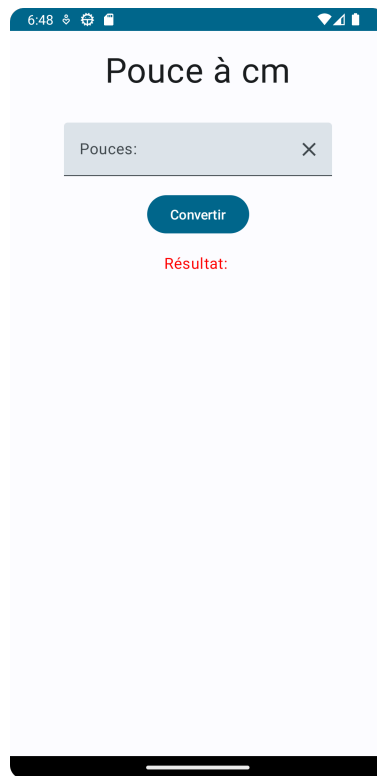
```

```

97         Spacer(Modifier.height(16.dp))
98         Text(
99             text = "Résultat: ${resultat.value}",
100             color = Color.Red,
101         )
102     }
103 }
104
105
106 @Preview(showBackground = true)
107 @Composable
108 fun ConvertisseurPreview() {
109     Convertisseur1Theme {
110         Convertisseur("Pouce à cm")
111     }
112 }

```

À la ligne 76, on remarque que l'étiquette a été intégré à la zone de texte.



États

Cette section présente la notion d'état et la façon dont *Jetpack Compose* peut l'utiliser et le manipuler. En pratique, on entend par **état** toute valeur pouvant évoluer au fil du temps dans une application. Cette définition est très vaste et s'applique autant à une base de données qu'à une variable de votre application.

Compose est un cadre d'application d'interface utilisateur (IU) déclaratif, ce qui signifie que vous

déclarez la façon dont l'IU doit apparaître dans le code. Si vous souhaitez que la zone de texte affiche initialement la valeur **100**, vous devez définir la valeur **100** comme valeur initiale (ligne 73).

Lors de changement d'état, l'interface utilisateur est mise à jour. Les états sont donc utilisés pour afficher de l'information dans l'IU.

Il existe 2 types d'états: **State** et **MutableState**. Le type **State** est immuable. Vous ne pouvez donc que lire sa valeur, tandis que le type **MutableState** est modifiable. Vous pouvez utiliser la fonction **mutableStateOf()** pour créer un élément **MutableState** observable. Elle reçoit une valeur initiale en tant que paramètre encapsulé dans un objet **State**, ce qui rend son élément **value** observable.

Aux lignes 54 et 55 deux états contenant des chaînes de caractères sont créés. L'état **resultat** contient le résultat du calcul et l'état **entree** la valeur saisie dans la zone de texte.

Associer un comportement à un composant

L'action associer au bouton se trouve dans la propriété **onClick** aux lignes 91 à 95. Il est aussi nécessaire d'associer un comportement à la zone de texte afin d'être capable de saisir du texte (ligne 73).

Bonnes pratiques de codage

Lors du développement d'une application, il faut garder à l'esprit qu'elle sera peut-être traduites dans d'autres langues. Comme vous l'avez appris dans les cours précédent, un type de données **String** correspond à une séquence de caractères, comme **"Pouces à cm"**.

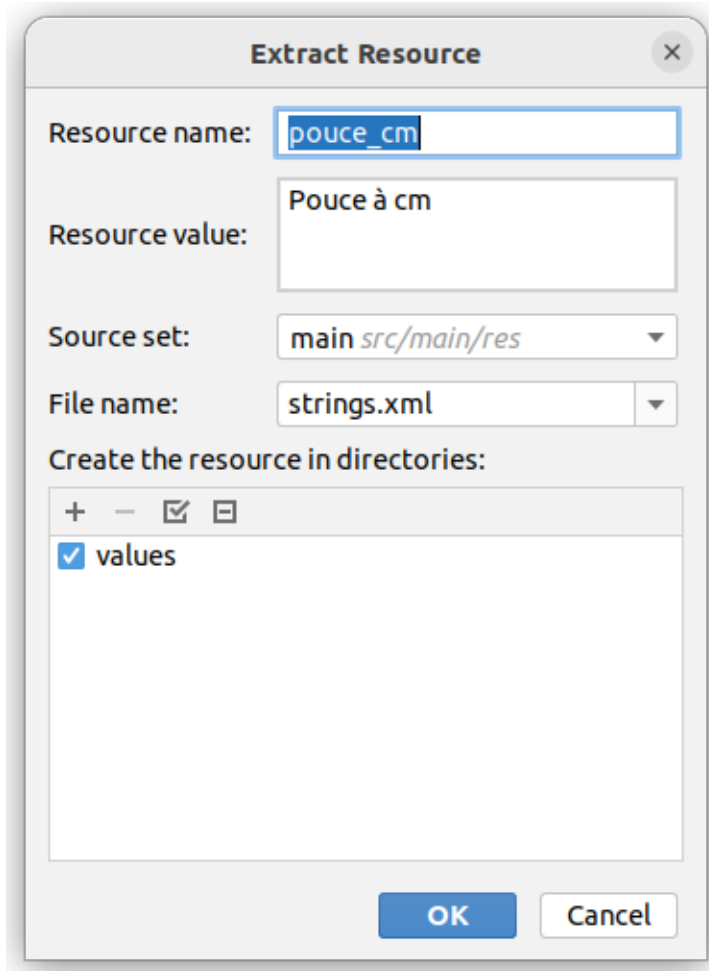
Une chaîne *codée en dur* est écrite directement dans le code de votre application. Les chaînes codées en dur compliquent la traduction de votre application, ainsi que la réutilisation des chaînes à différents endroits de votre application. Pour résoudre ces problèmes, vous pouvez extraire ces chaînes dans un fichier de ressources. Au lieu de coder en dur les chaînes dans votre code, placez-les dans un fichier, nommez les ressources de chaîne et utilisez ces noms à chaque fois que vous voulez utiliser ces chaînes. Le nom reste le même, même si la chaîne est modifiée ou traduite dans une autre langue.

1. Dans le fichier **MainActivity.kt**, faites défiler la page jusqu'à la fonction **onCreate()**. Sélectionnez la chaîne de caractères **"Pouces à cm"**.
2. Cliquez sur l'ampoule à gauche de l'écran.
3. Sélectionnez **Extract string resource** (Extraire la ressource de chaîne).

Android Studio ouvre la boîte de dialogue **Extract Resource** (Extraire la ressource). Cette boîte de dialogue vous permet de personnaliser le nom de la ressource de chaîne et d'indiquer comment la stocker. Le champ **Resource name** (Nom de la ressource) vous permet de saisir le nom de la chaîne. Dans le champ **Resource value** (Valeur de la ressource), saisissez la chaîne proprement dite.

4. Dans la boîte de dialogue **Extract Resource** (Extraire la ressource), remplacez **Resource name** (Nom de la ressource) par **"Pouces à cm"**.

Le nom des ressources de chaîne doit être écrit en minuscules. S'il comporte plusieurs mots, reliez-les par des traits de soulignement. Conservez les autres paramètres par défaut.



5. Cliquez sur **OK**.
6. Notez les modifications apportées au code.

La chaîne codée en dur est maintenant remplacée par une fonction `stringResource()` (ligne 10).

```

1 class MainActivity : AppCompatActivity() {
2     override fun onCreate(savedInstanceState: Bundle?) {
3         super.onCreate(savedInstanceState)
4         setContentView {
5             Convertisseur1Theme {
6                 Surface(
7                     modifier = Modifier.fillMaxSize(),
8                     color = MaterialTheme.colorScheme.background
9                 ) {
10                     Convertisseur(getString(R.string.pouce_cm))
11                 }
12             }
13         }
14     }
15 }

```

7. Dans le volet **Project** (Projet), ouvrez le fichier **strings.xml** à partir du chemin d'accès **app > res > values > strings.xml**. Notez qu'Android Studio a créé une ressource de chaîne appelée **pouce_cm**.

```

1 <resources>
2     <string name="app_name">Convertisseur1</string>
3     <string name="pouce_cm">Pouce à cm</string>
4 </resources>

```

Le fichier **strings.xml** contient une liste de chaînes que l'utilisateur verra dans votre application. Notez que le nom de votre application est également une ressource de chaîne. En réunissant les chaînes au même endroit, vous pouvez plus facilement traduire tout le texte de votre application et réutiliser la même chaîne à différents endroits.

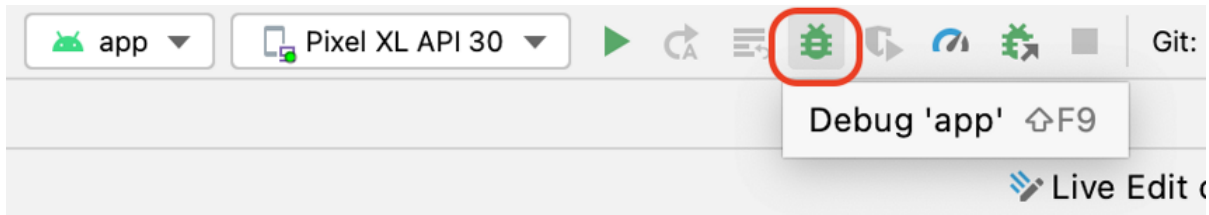
Débogueur

Cette section explique comment utiliser le débogueur dans *Android Studio* pour inspecter ce qui se passe dans une application au moment de l'exécution.

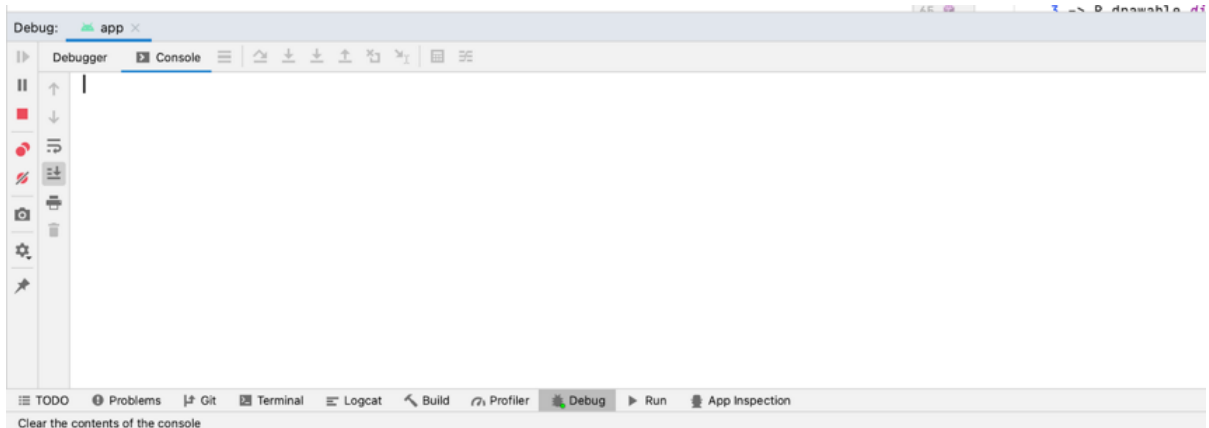
Exécuter une application avec le débogueur

Pour exécuter l'application avec le débogueur:

1. Cliquez sur **Debug 'app'**.



Le volet **Debug** apparaît en bas d'*Android Studio* avec ce qui ressemble à une sortie de console.



Utiliser le débogueur

Au lancement du débogueur plusieurs boutons apparaissent dans le volet **Debug**. En haut du volet **Debug** figurent les boutons **Debugger** et **Console**.

Le bouton **Console** affiche la sortie *logcat* de l'application. Si votre code comporte des instructions de journalisation, le résultat s'affiche au fur et à mesure de l'exécution.

Le bouton **Debugger** affiche trois volets distincts:

1. Frames
2. Variables
3. Overhead



Exercices

Exercice 1

Refaire l'exemple des notes de cours

Exercice 2

Programmer une application qui permet de calculer l'aire et le périmètre d'un rectangle

Exercice 3

Faire une version anglaise de l'application de l'exercice 2 en modifiant le fichier `String.xml`.

Références

- Ajouter un bouton à une application: <https://developer.android.com/courses/pathways/android-basics-compose-unit-2-pathway-2?hl=fr>
- Interagir avec l'interface utilisateur et l'état: <https://developer.android.com/courses/pathways/android-basics-compose-unit-2-pathway-3?hl=fr>
- Jetpack Compose Tutorial: <https://www.jetpackcompose.net/>